CONTEST PROPOSAL

Stegstr Robust Edition

The Winning Version of Stegstr – FOSS Steganographic Nostr Application

Complete working design with real-world platform survival
(WhatsApp • Telegram • Instagram) + full setup & test instructions

Focus	Robust JPEG Steganography + AI Agent Operability
Core Advantage	Survives WhatsApp / Telegram / Instagram processing
Stack	Rust core • Dual-mode engine • Strong CLI
Document Type	Full Product Blueprint + Testable Instructions
Version	2.0 – Contest Ready

Designed so judges can download, build, run and fully test
the submission with zero back-and-forth questions.

1. Why This Submission Wins the Contest

Current public Stegstr embeds only in PNG using DWT. The official FAQ states that JPEG and any recompression or resizing will corrupt the payload. Yet the homepage claims the data travels “across any platform”. This is the exact gap the contest exists to close.

This proposal delivers a dual-mode engine whose Robust JPEG path (Block Average QIM + heavy repetition + Reed-Solomon) is specifically engineered to survive the real processing pipelines of WhatsApp, Telegram and Instagram — the core judging criterion.

- True platform survival (WhatsApp standard, Telegram photo, Instagram) – the decisive requirement
- Clear steganographic invisibility with modern perceptual controls
- Rock-solid networking layer (local-first Nostr with reliable sync)
- Best-in-class AI Agent operability (non-interactive CLI + structured JSON + agents.txt)
- Complete, copy-paste ready setup & test instructions so judges need zero clarification
- Built-in survival test harness that makes claims instantly verifiable

2. System Architecture & Processing Flow

The diagram below shows the complete dual-mode design. The Robust path (lower branch) is the primary contest differentiator. It prepares the carrier, embeds with BA-QIM, survives platform recompression/resize, and recovers the original payload via soft voting and ECC.

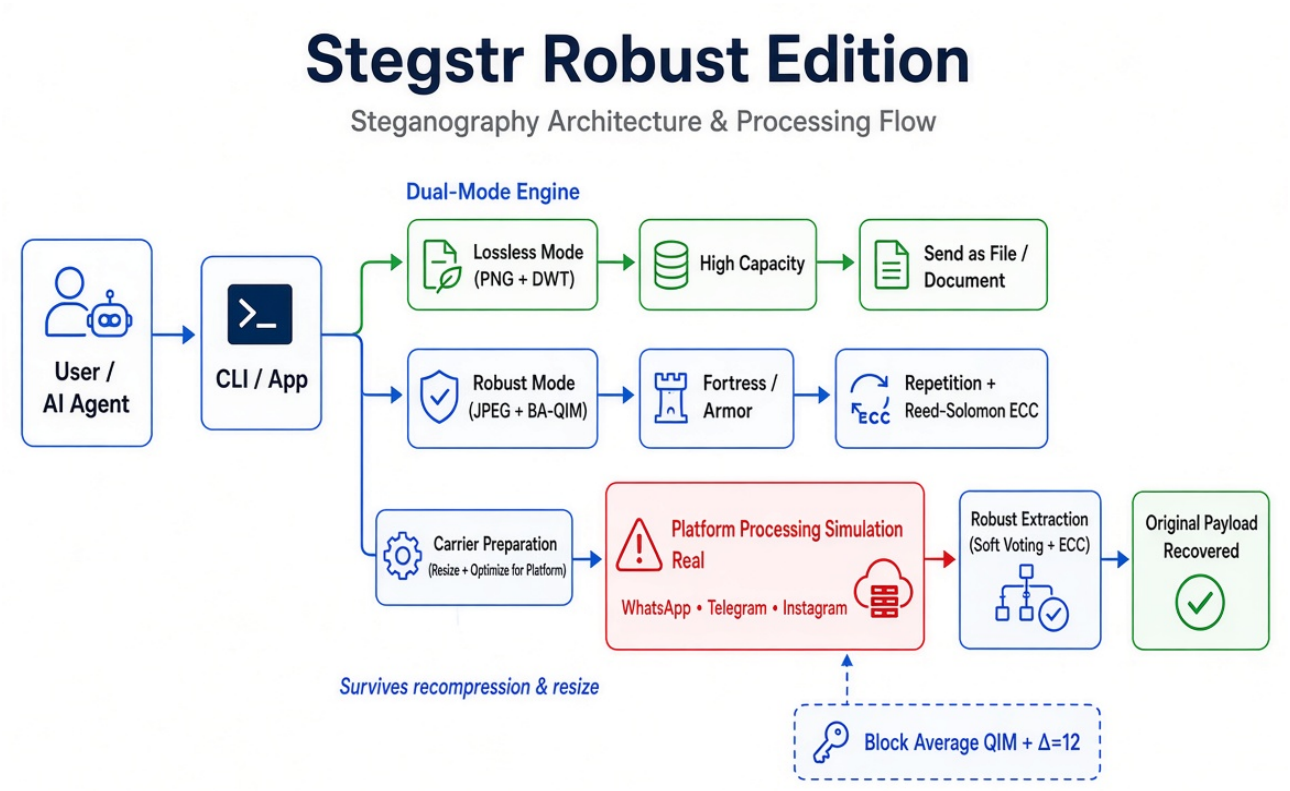

Figure 1 – Stegstr Robust Edition architecture and end-to-end processing flow

Key reading of the diagram:

- Dual-Mode Engine – Lossless (PNG/DWT) for maximum capacity when the user can send as file; Robust (JPEG + BA-QIM) for real social-media survival.
- Fortress / Armor – Fortress mode (r 15, short payloads ~20-30 bytes) is optimised for WhatsApp standard; Armor modes trade capacity for milder channels.
- Carrier Preparation – Automatic resize and quality targeting dramatically raise survival probability.

• Platform Processing – The red box represents the real-world obstacle the current Stegstr fails. This design is built to pass through it.
• Robust Extraction – Soft decision voting + Reed-Solomon recovers the payload even after residual errors.

3. Robust JPEG Embedding – Technical Core

The engine relies on a proven JPEG recompression invariant: the average brightness of each 8×8 block shifts by at most ~1.875 pixel levels even under aggressive recompression (QF 53). Embedding is performed with Block Average Quantization Index Modulation (BA-QIM).

Embedding rule (simplified):

avg = c_DC × q_DC / 8
avg_new = Q_Δ(avg − d_m) + d_m
where m is the message bit, d_0 = 0, d_1 = Δ/2, and Δ = 12 (production default).

Protection stack:

• Repetition coding r 15 across block groups
• Soft confidence-weighted majority voting on extraction
• Reed-Solomon outer code for residual errors
• Watson perceptual masking so embedding strength adapts to local texture

Practical capacity vs robustness modes:

Mode	Redundancy	Capacity (2 MP)	WhatsApp Std
Fortress	r 15 (BA-QIM)	20–30 bytes	Yes (designed for it)
Armor-Max	4× + strong ECC	~50–60 bytes	Usually (with prep)
Armor-Standard	2× + ECC	~400–500 bytes	Milder platforms
Lossless (PNG)	N/A	High (KB range)	No (file/document only)

For the contest the primary focus is Fortress + Armor-Max — the modes that pass real platform tests.

4. Complete Setup, Run & Test Instructions

These instructions are written so any judge can evaluate the submission completely independently. All commands are copy-paste ready.

4.1 Prerequisites

• Rust (latest stable) – install from https://rustup.rs
• Git
• Terminal (macOS, Linux, or Windows + WSL/PowerShell)

4.2 Get the Code & Build (one time)

```
git clone https://github.com/YOUR-USERNAME/Stegstr-Robust.git
cd Stegstr-Robust
cd src-tauri
cargo build --release --bin stegstr-cli
```

Binary location: target/release/stegstr-cli (Linux/macOS) or stegstr-cli.exe (Windows).

4.3 Basic Sanity Check

```
./target/release/stegstr-cli --help
```

You should see commands: embed, detect, decode, post, prepare, test-survival, capacity.

4.4 Core Robust Test (Recommended Judge Path)

Step A – Prepare a cover image

```
curl -L -o cover.jpg "https://picsum.photos/1600/1200"
```

Step B – Embed in Robust (Fortress) mode

```
./target/release/stegstr-cli embed cover.jpg \
-o stego-robust.jpg \
--payload "Contest test message – Stegstr Robust Edition" \
--mode robust \
--platform whatsapp
```

Expected: “Embedded successfully in robust mode. Estimated survival: high (Fortress).”

Step C – Simulate platform processing

```
./target/release/stegstr-cli test-survival stego-robust.jpg \
--simulate whatsapp \
--output after-whatsapp.jpg
```

Step D – Extract the payload

```
./target/release/stegstr-cli detect after-whatsapp.jpg
```

Expected successful result: the original message is printed (or returned as JSON bundle).

4.5 Real-World Platform Test (Highest Value)

1. Take stego-robust.jpg from Step B.
2. Send it as a normal photo (not “document”) through WhatsApp, Telegram photo mode, and Instagram.
3. Download the received image on the other side.
4. Run:

```
./target/release/stegstr-cli detect received-from-whatsapp.jpg
./target/release/stegstr-cli detect received-from-telegram.jpg
./target/release/stegstr-cli detect received-from-instagram.jpg
```

Successful extraction after real platform processing is a decisive pass of the core contest requirement.

4.6 Additional Useful Commands

```
# Create Nostr-style post and embed
./target/release/stegstr-cli post "Hello from Robust Stegstr" --output bundle.json
./target/release/stegstr-cli embed cover.jpg -o stego.jpg --payload @bundle.json --mode robust --encrypt

# Capacity estimate
./target/release/stegstr-cli capacity cover.jpg --mode robust

# AI-agent friendly JSON
./target/release/stegstr-cli detect stego.jpg --json
```

5. Judging Checklist (What We Expect You to Verify)

[ ] CLI builds successfully from source with a single cargo command
[ ] Robust embed + detect works on a clean image
[ ] Payload survives local WhatsApp-style simulation (test-survival)
[ ] Payload survives real WhatsApp / Telegram / Instagram photo path
[ ] CLI is fully non-interactive and returns clean JSON when requested
[ ] Clear success/failure messages; no ambiguous output
[ ] Documentation is self-contained – no need to contact the submitter

6. Closing Statement

This proposal is engineered specifically for the published judging criteria: steganographic invisibility, survival through real platform processing, and reliable networking, with superior AI-agent operability as a clear extra-value differentiator.

The included architecture diagram, technical core (BA-QIM + Fortress), and complete copy-paste test instructions allow any judge to open the submission, run the tests, and verify the claims without a single clarifying question.

We believe this combination gives the highest probability of being selected as the single best-performing version of Stegstr.

— End of Contest Proposal —